Transcript — The Estimator

Deck: `the-estimator.pdf`. Nói slide 1–13; A1–A3 chỉ mở khi được hỏi.

Slide 1

Thanks for having me. I'm Thong. I'll walk through how I built The Estimator with AI: framing the problem, coordinating implementation, and checking the result. The initial build was October 16–19. The deeper missing-data correction came after Fest Night.

Slide 2

I scoped the challenge to software-services requests: Sales prepares an estimate for technical review. The output is person-days, not a delivery date. I chose fifteen scope features as a starting point. Without company history, I used 240 synthetic projects to test the mechanism—not prove real-world accuracy. Learning from real history would require consistent scope and actual-effort records.

Slide 3

The language model reads the request and extracts scope with source quotes. Present, explicitly absent, and unknown are different states. A separate calculator produces an effort range. Follow-up questions resolve missing scope, and completed work can feed the next update. A person reviews the result before making a commitment.

Slide 4

The initial specification defined the feature names, true/false/null behavior, and API response before implementation. These contracts gave coding agents bounded tasks. An offline stub let other layers develop without live calls. The gain was independently testable modules; the cost was more schema and integration work. Offline tests still didn't establish live extraction quality.

Slide 5

OpenCode was my coding-agent environment; DeepSeek was the extraction provider inside the app. Data/database and core services were implemented with separate file ownership and shared interfaces, then connected through the API and UI. QA exposed ID collisions, invalid actual effort, and deployment assumptions. Parallel generation still needed integration checks and regression tests. Deployment was consolidated into one service.

Slide 6

One interaction I remember was asking the agent to evaluate the estimator on TAWOS. It pointed out that the available fields and labels didn't support the full evaluation. I asked: even if we can't test the whole solution, which part can we still test? That's a paraphrase of the interaction. The goal was a narrower useful experiment, not forcing a positive result.

Slide 7

The experiment used 5,850 Jira issues from 39 projects, with an offline keyword extractor and an estimator refit on story points. On unseen projects, it essentially tied the median baseline. It exposed limited vocabulary coverage and a person-day floor unsuitable for story points. The negative result was useful diagnostically, but didn't validate the full estimator.

Slide 8

The repository analyzer produced different local and cloud scores. The local search had found a coverage report inside a nested demo repository. It was valid XML about the wrong repo. The fix checked whether report file references resolved in the analyzed checkout; otherwise coverage became unknown. A regression test and matching local/cloud results verified the targeted correction.

Slide 9

After Fest Night, extraction-path testing and external review challenged the missing-data assumption. About 25 percent of feature cells were positive overall, but only 4.1 percent among extraction-null cells. Sampling at the overall rate invented scope. A null-conditioned prior reduced error from 49.64 to 20.48 days. Compared with zero-fill, the main gain was coverage, not a large MAE improvement. The cost is maintaining another artifact. Working code can still implement the wrong assumption.

Slide 10

I separate implementation evidence from business evidence. The recorded suite has 242 offline tests, and real-provider extraction was evaluated separately. Synthetic replay helps diagnose the system, but doesn't establish superiority over humans: extracted-feature MAE is 20.48 versus 18.26 for the corrected simulated-team baseline. Real handling-time savings, adoption, and accuracy still need a pilot.

Slide 11

If I started again, I'd test the real extraction path and simplest baselines earlier. I'd keep the first slice smaller and seek user feedback sooner, rather than expand optional experiments. I'd retain the contracts because they helped isolate failures. More generated code isn't automatically more useful progress.

Slide 12

At Anduin, I'd reuse the method, not assume this estimator is the tool you need. I'd observe one workflow, agree a baseline and owner, then build a small slice using AI, a script, SQL, or rules. We'd agree the data boundary and pilot with review. We'd measure total handling time and quality, including rework, then keep, simplify, or stop—and document ownership and fallback.

Slide 13

This project shows how I frame tasks, build with AI, and verify the result. That includes accepting feedback and changing direction when evidence contradicts an assumption. That's the way of working I'd bring to the team. Thank you—I'm happy to open the code or discuss any of these examples.